A Modular SystemC RTOS Model for Uncertainty Analysis

Lorenzo Lazzara(✉), Giulio Mosé Mancuso, Fabio Cremona, and Alessandro Ulisse

United Technologies Research Center, Rome, Italy
{lorenzo.lazzara,giuliomose.mancuso,fabio.cremona, alessandro.ulisse}@utrc.utc.com

Abstract. Nowadays the complexity of embedded systems is constantly increasing and several different types of applications concurrently execute on the same computational platform. Hence these systems have to satisfy real-time constraints and support real-time communication. The design and verification of these systems is very complex, full formal verification is not always possible and the run-time verification is the only feasible path to follow. In this context, the possibility to simulate their behavior becomes a crucial aspect. This paper proposes a SystemC modular RTOS model to assist the design and the verification of real-time embedded systems. The model architecture has been designed to capture all the typical functionalities that every RTOS owns, in order to easily reproduce the behavior of a large class of RTOS. The RTOS model can support functional simulation for design space exploration to rapidly evaluate the impact of different RTOS configurations (such as scheduling policies) on the overall system performances. Moreover the model can be used for software verification by implementing specific RTOS APIs over the generic services provided by the model, allowing the simulation of a real application without changing any instruction. The proposed approach enables the user to model non-deterministic behaviors at architectural and application level by means of probabilistic distributions. This allows to assess system performances of complex embedded systems under uncertain behavior (e.g. execution time). A use case is proposed considering an instance of the model compliant with the ARINC 653 specification, which requires spatial and temporal segregation, and where typical RTOS performances are assessed given the probability distributions of execution time and aperiodic task activation.

Keywords: Real-time operating system model · SystemC · Uncertainty quantification · Statistical model checking

1 Introduction

Virtual engineering techniques are increasingly popular in several application domains. Models of the system components (seen as "virtual components") are

R. Chamberlain et al. (Eds.): CyPhy 2019/WESE 2019, LNCS 11971, pp. 3–27, 2020.
https://doi.org/10.1007/978-3-030-41131-2_1

used to assess the behavior and performance of a subsystem unit without the need of a physical prototype. This capability is usually exploited (1) in the specification of the subsystem components to compare different design alternatives and select the best design option; (2) in the verification flow, where the satisfaction of requirements can be assessed on a virtual prototype. This approach allows to start the verification process earlier in the design flow anticipating errors that would otherwise propagate along the chain up to the final implementation [5]. System Level Modeling [11,21] seems to be the only feasible way to face the growing design complexity of modern embedded systems. It allows the modeling of critical embedded platform details at high abstraction levels. This enables faster exploration of the design space at early stages and ease the verification of software integration before having access to the real hardware. However existing System Level Design Languages (SLDL) and methodologies do not support a proper modeling of a full real time operating system at higher abstraction level. The Real-time operating system is a critical software component that orchestrates all the application execution in the embedded platform. Hence almost all the timing properties of the platform are dependent from the RTOS. For this reason in the current paper we aim to fill the gap of the current SLDL proposing a modular SystemC generic real-time operating system model. The proposed RTOS model takes into account different functional aspects, providing real-time scheduling and other typical services necessary for real-time software simulation. The modularity of the proposed model ease the configuration of the possible instances enabling functional simulation for design space exploration. The proposed model also provides the capability to implement specific RTOS services allowing a fast integration of the target platform code for software-in-the-loop (SIL) simulations. The proposed RTOS model also considers stochastic behaviors naturally appearing in the real embedded platforms. For example nowadays the computational platform presents complex features like caching, pre-fetching, pipelining, DMA, and interrupts which allow to improve the performance of the system but can lead to a higher degree of uncertainty. In this scenario the timing characteristic of the system can be very unpredictable and a rigorous and precise timing analysis can be very difficult to conduct [4]. In addition, complex embedded system operates in an environment that is intrinsically non-deterministic. Two kinds of non-determinism can be distinguished in embedded systems [19]: pure non-determinism and probabilistic non-determinism. Pure non-determinism is for example the non-determinism due to the interaction with the environment (sensor data input) that is highly unpredictable. Probabilistic non-determinism is appropriate whenever a statistical representation of the phenomenon well represent is occurrence. We believe that the response time may lie in this category. In the current work we will focus on probabilistic non-determinism allowing a stochastic characterization of the main kernel operations by defining their behavior in terms of probability distributions. The proposed approach is validated considering a data acquisition application running on an RTOS model compliant with the ARINC 653 specification [2]. The standard ARINC 653 aims at support the integrated modular avionic framework providing a strict and robust

time and space partitioning environment with the definition of a common API (called APEX) between the software applications and the underlying operating system.

2 Related Works

Different techniques have been proposed for high abstraction level RTOS modeling and simulation. One of the early techniques was presented in [23] which uses a specific RTOS model and simulation engine. It allows an highly accurate simulation but limits the design space exploration and the software verification to the specific RTOS. Approaches based on System Level Domain Languages has been proposed in order to provide a general RTOS model able to simulate different RTOS instances. A fully configurable RTOS model was proposed in [15] which targets a functional model simulation. This work models both the behavior and the time aspects of the RTOS for both singlecore as well as multicore platform architectures. It allows the configuration of the model in terms of HW resources and RTOS functionality, moreover it integrates a timed simulation by inserting back-annotation at different level of granularity. The work described in [17] reports a time accurate RTOS model for POSIX compliant applications. It allows the simulation of application source code integrated into the RTOS model to be simulated with dynamically execution time estimation. The work described in [9] present a customizable RTOS with different modules interacting with each other with the definition of a generic services interface towards the task which ease the integration of various RTOS APIs. The approach presented in this paper is based on SystemC [1] similarly to the one proposed in [9,15,17]. In particular those papers address the case in which the designer is able to configure the RTOS model in terms of different functional aspects like scheduling policies, communication mechanisms and other typical RTOS features. However they do not address the possibility to model hierarchical architectures with multi-level schedulers. This type of architecture are important for safety critical systems (i.e. automotive, avionics) where applications usually execute in a temporal and spatial segregated environment. The work in the [17] provides a POSIX API from which the user can interact and run application code. This is an interesting feature, however this does not allow to model other kind specific RTOS APIs. The annotation mechanism discussed in [14] sec. 2.3 consider an annotation for each line of code through the use of a DELAY function. Our approach of annotation is similiar, we introduce a function which allow to simulate the task physical running time on a CPU with the possibility of stopping the time in order to mimic preemption, the behavior of this function is similiar to the one presented in [24]. The model we propose herein takes into account the variability present in the system providing a framework for uncertainty analysis. Several works have been focusing on the uncertainty characterization and propagation for Embedded Systems. The work in [25] and in [26] mainly focused on a formal software representation of the uncertainty in order to support Model Based Testing (MBT) techniques. The work in the [8] and in [7] instead targeted the

formal verification of the application code (written in SystemC) by using affine arithmetic. Although those methods provide a formal framework for the verification under uncertainty, they may not scale well with complex application code. The rest of the paper is organized as follows: in Sect. 3 we introduce the model architecture and we will give more details about the software components. In Sect. 4 we introduce the uncertainty analysis able to run on the proposed RTOS model. Finally in Sect. 6 we will show some experimental results on a typical ARINC 653 data acquisition application. The conclusions and the interesting future work will be exposed in Sect. 7.

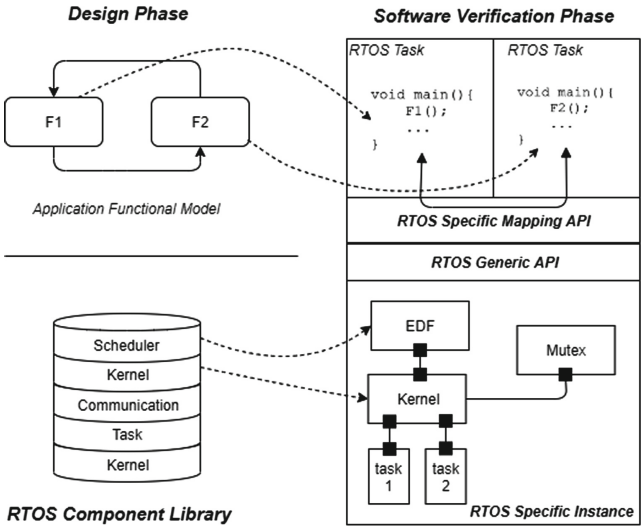


Fig. 1. Conceptual architecture and workflow.

3 Real Time Operating System Model Architecture

In the following section we present the high level model architecture of a SystemC-based System-Level Real-Time Operating System (RTOS) software simulation framework. The main idea behind the proposed approach is the definition of a *Component Library* from which the user can take components and build specific RTOS model instances. The structure of the overall RTOS model can be decomposed in three main parts: the *Generic RTOS Component Library*, *Generic RTOS API* and a *Specific RTOS Mapping API*. A graphical representation of the architecture and a conceptual workflow is depicted in Fig. 1. The generic component library is divided into functional categories i.e. kernel, scheduler, task, communication and synchronization. The user can instantiate a

specific RTOS model instance interconnecting specific components from different categories. Components belonging to the same category implement a standard execution interface that allows to easily interchange different blocks with the same interface but with different implementation. This modular approach ease the evaluation of the impact of different implementation choices during the design space exploration. The generic API exposes externally generic RTOS services from all the categories e.g. part of the task management services are exposed in Fig. 2. The specific mapping API focuses on the implementation of specific RTOS services. For example the ARINC 653 and the AUTOSAR specifications detail a specific API (and their semantics) that a real-time operating system should implement. The main purpose of the mapping API is to map the specific services into the generic services. The implementation of a specific API ease the software integration verification phase. As depicted in Fig. 1, our model may enable the validation and the verification of the application at different level of abstraction. For example at the early design stage (e.g. functional model), the generic APIs can be used to validate the preliminary application-RTOS task mapping (top part of Fig. 1). Once the target platform has been selected, the mapping API can be implemented in order to enable software-in-the-loop verification. The interaction between the main components of the system (kernel, scheduler and task) is achieved through SystemC *ports* and *exports* which require the definition of a common interface in order to communicate each other. The interfaces have been defined in a way that they are as much as possible independent from the particular implementation of the services. This architecture allows to decouple the declaration of the system calls (defined at the interface) with their actual implementation in order to guarantee and ease the design exploration by enabling the replacement of a component with another without changing any other component in the overall model. The proposed high-level system architecture is presented in Fig. 2.

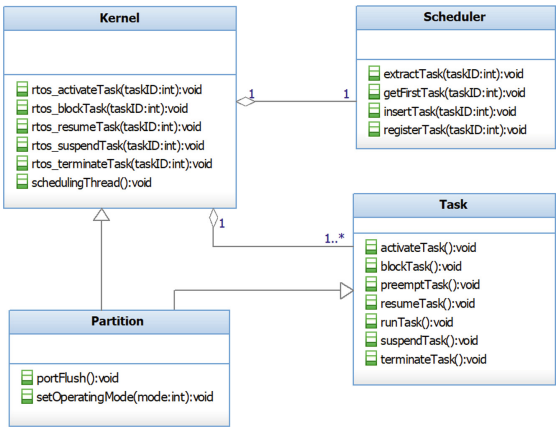


Fig. 2. Simplified system class diagram.

The RTOS model has been integrated into the *Desyre* [5] framework which is a SystemC-based virtual prototyping environment developed at the *United Technologies Research Center*. However the model is based on open standards such as SystemC and IPXACT allowing its integration in any framework which supports those standards.

3.1 Scheduler Model

In all the real-time operating systems the scheduler is in charge of maintaining the correct execution order between the tasks. More formally at each instant in time the scheduler has to decide which is the running task and has to build a queue of available tasks ready to run. Our aim is to model a generic real-time scheduler with different scheduling policies. When tasks are registered within the scheduler, a relative `TaskSchedulingModel` object is instantiated, which contains the scheduling parameters and ID of the task. This approach allows the separation of the task parameters (like period, deadline, wcet, etc.) that are contained in the task class, from the scheduling parameters that are contained in the `TaskSchedulingModel` and which are defined specifically depending on the chosen scheduling algorithm. The ready queue is ordered based on three parameters defined in the `TaskSchedulingModel`. The `priority` is the first level sorting parameter and it depends on the scheduling algorithm used. In order to have a generic priority which would capture both time-dependent scheduling algorithms (e.g. EDF) and fixed priority scheduling algorithms (e.g. FP, RM), this parameter is modeled as a *sc_time* SystemC variable. The priority parameter is used to sort the tasks in ascending order (smaller is the priority value, higher is the scheduling priority) inside the ready queue. If tasks have the same priority value the `insertion time` is used to give higher priority to the oldest tasks in the ready queue, hence following a FIFO sorting principle. The `task ID` is used when modeling an ideal RTOS without system overheads, having therefore the possibility of having tasks with same priority inserted at the same time. The model allows to select among several scheduling policies. *First Come First Served (FCFS)* is implemented exploiting the second level sorting parameter (*insertion time*) to order the tasks in the ready queue. In order to implement the time-sharing preemptive mechanism of the *Round-Robin (RR)* scheduling policy, an event is used in order to trigger the kernel and force a rescheduling whenever a time quantum expires. For *Rate Monotonic (RM)*, when the task is registered within the scheduler, its priority (period) is used to construct the relative `TaskSchedulingModel`. Hence the priority value is assigned only one time (static) and is never changed during execution. In *Earliest Deadline First (EDF)*, since the priority value can not be assigned statically, the task deadline value is retrieved through the interface with the kernel whenever the task is inserted in the ready queue. For *Fixed Priority (FP)*, when the task is registered within the scheduler, its priority is retrieved from a priority-pool inside the scheduler and it is used to construct the relative `TaskSchedulingModel`. The *Partition Scheduler* implements a partition scheduling policy which is a predetermined, repetitive with a fixed periodicity, called *major time frame* (MTF), scheduling

algorithm. The scheduling parameters, the major time frame, the offset and duration of each partition window are set during the system configuration. The offset of the partitions is used as a priority in the `TaskSchedulingModel` to sort the partitions in the ready queue, the duration instead is used like the slice time in the RR scheduler, i.e. it defines the next preemption time. Being a time-driven scheduler the interaction with the kernel is similar to that described for the RR. This kind of partition scheduling is used in all the RTOS systems compliant with the ARINC 653 standard. An example of partition scheduling will be presented in the later sections (see Fig. 7).

3.2 Kernel Model

The RTOS Kernel module models all the typical software mechanism of an RTOS, such as: task scheduling, task interaction and synchronization. The kernel model allow to ensure that the execution of the tasks is serialized following the order of the selected scheduling policy. In order to achieve this behavior, all the tasks wait on specific SystemC events which are only released by the kernel. Hence, during scheduling the kernel retrieves the first task from the ready queue and execute it by triggering its event. If there is not a candidate task (ready list is empty), the kernel just waits until a ready task is available. Each task is associated with a task descriptor class that is a data structure containing all its static and run-time information, i.e. task ID, period, deadline and task state. The interaction between the kernel and the task is performed through the data exposed by the task descriptor. The RTOS model supports both periodic and aperiodic tasks. A periodic task at end of its job will call the task wait cycle method in order to wait for its next release point while an aperiodic task will call the terminate method in order to kill its instance. The *Task State Machine* is the basis of both multi-tasking management and scheduling services in the RTOS kernel model. Typical multi-tasking primitive functions include creating tasks, activating tasks, suspending tasks, blocking tasks, resuming tasks, and terminating tasks. These functions control the state transitions of tasks during their execution. In the Fig. 3 is depicted the RTOS task state machine that has five basic states: `dormant`, `running`, `ready`, `suspend` and `blocked`. During the RTOS execution a task can be at only one state:

- `running`: in a uniprocessor system, only one task can enter this state and execute at each time instant. If the `running` task is preempted, then it enters the `ready` state.
- `ready`: tasks at this state are eligible for execution, but cannot execute immediately as another task is currently at the `running` state. All `ready` tasks are organized in the ready queue by the scheduler according to various scheduling policies. During a rescheduling the kernel retrieve the first task in the ready queue; if this new task is different from the `running` task (if there is one), the `running` task is preempted and the new task is dispatched.
- `blocked`: tasks enter the `blocked` state when accessing an (empty) blocking resource or when they explicitly wait for a timer. Each `blocked` task is organized in a waiting queue relative to the blocking resource. Usually a timeout

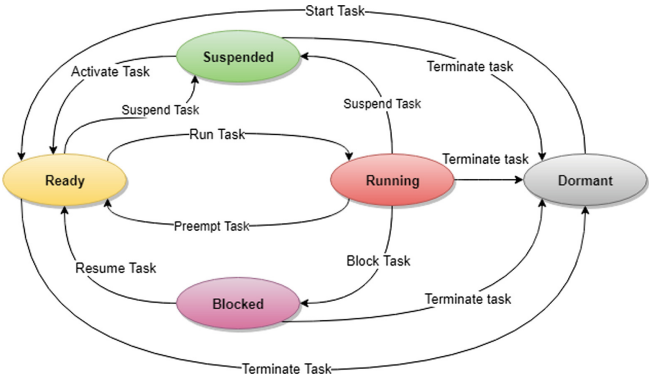


Fig. 3. Task finite state machine.

period can be specified in order to automatically unblock and put in the `ready` state the task.
- `suspended`: similarly to the tasks in the `blocked` state, tasks in the `suspended` state cannot be selected to enter the `running` state. The tasks only enter the `suspended` state when the `TaskSuspend()` service is explicitly called. The task exit the `suspended` state when the resume or activate service is called.
- `dormant`: a task is said to be in the `dormant` state when is created, but has not yet been started, or when its execution is completed.

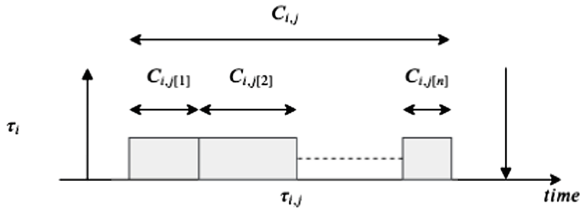
Application tasks need to synchronize and share data, in order to cooperate with each other properly. In the RTOS we can distinguish between *synchronization methods* and *communication methods* where the former is used mainly to coordinate the execution orders of involved tasks, while the latter can explicitly exchange data between tasks. Unlike the system components previously described, communication services do not interact with the system through SystemC *ports* and *exports*. The kernel contains a pool of pointers to communication objects which are discriminated through their communication ID that is assigned to each communication object when it is instantiated. Modeling remains general thanks to the introduction of a common generic interface; each communication object will have a specific implementation of this interface. Communication services are accessed at task level through the interface with the kernel. Inter-task communication methods are exploited via message passing. The communication mechanisms can be fully configured in order to have message queues or shared variables (single instance message), blocking or non-blocking mechanism, queuing policy and other meaningful parameters. The inter-task synchronization can be achieved through the use of counting semaphores and events. Semaphores are used to provide controlled access to resource, while events support control flow between tasks by notifying the occurrences of conditions to waiting tasks. In order to allow the modeling of temporally segregated RTOS, an extension of the kernel model has been done to comprehend the concept of a partitioned RTOS. A partition is a schedulable entity where one or more tasks concurrently

execute; hence, the partition has been modeled, by implementing both the kernel and task interface, as a kernel that can be in turn scheduled. When the partition is selected to run the partition-level scheduler is activated and tasks execute as they are running on a normal kernel. The partition execution can be preempted and in turn if there is a running task, it is also preempted in order to allow the execution of a new partition and the tasks it contains. This run-time behavior is modeled through a SystemC `SC_THREAD`. As the kernel manages scheduling and communication between tasks, an entity is needed to manage the partitions. The *Supervisor* is a simple kernel that handles partition scheduling and dispatching, as well as inter-partition communication. The supervisor allows the exchange of information between different partitions by means of communication ports. Ports can have different configurations like blocking/non-blocking mechanism, possibility of message queuing with the related queuing policy and direction of transfer. Whenever a partition switch takes place, the outgoing ports, of the currently running partition, are checked for new data available. In case of data transfer the partition inform the supervisor which takes charge of the inter-partition communication transferring data to the destination ports in one or more partitions. The supervisor, together with the partition scheduler, allows to achieve time segregation by limit the processing time assigned to each partition. Whenever the end of a partition time window is reached the supervisor is triggered, it preempts the current running partition and selects the next partition to run. The supervisor and the partitions allow to instantiate an hierarchical RTOS with multiple level of scheduling; moreover each partition can represent a different instance of a RTOS kernel by modifying the attached scheduler or its behavior.

3.3 Functional and Timing Task Model

An important characteristic of the real-time systems is their tight dependency between functional and timing characteristic of the task. It is well known that the correct execution of a real-time application is not only determined by the correct functional output, but it needs to be provided within some time bounds (deadline). It is important then that our RTOS model properly captures both the functional and timing characteristics of the tasks running on top of it. From a functional point of view the proposed RTOS is able to execute the original RTOS task implementation with minor modification (time annotation). This is an extremely useful capability in a verification workflow where the code under test does not need to be modified moving between different validation/verification steps. This is achieved by mapping the generic RTOS APIs to the specific target RTOS services. Indeed when a new real-time operating system (e.g. FreeRTOS [3], DEOS [6], VxWorks [23]) is selected as suitable for the final system implementation, the modeler needs to implement the mapping code between the generic RTOS APIs and the specific RTOS services. An example of this mapping is detailed in Sect. 3.4 for the ARINC 653 specification. An important capability of the RTOS model is also to advance the virtual simulation time corresponding to the virtual execution of the application code. The timing characterization of the

tasks is of crucial importance to verify all the timing RTOS performances such as overall tasks schedulability, response time, jitter and so on. There are several approaches to characterize the timing behavior of the task body. A very nice survey on all those techniques can be found in [22]. Out of the scope of the current work is to perform a rigorous execution time analysis of the application code. The main objective however is to be flexible enough to enable most of the timing analysis techniques. We propose a timing task model where the overall execution time is divided into several execution chunks as depicted in Fig. 4a. The zth execution chunk of the jth job instance of τ_i is denoted by $\tau_{i,j[z]}$. The execution time of $\tau_{i,j[z]}$ is denoted by $C_{i,j[z]}$. The total execution time is defined by $C_{i,j} = \sum_{z=1}^{n} C_{i,j[z]}$. In case of fixed execution time among all the task jobs, we remove the index j relative to the task job, e.g. $C_{1,1} = C_{1,1[1]} + C_{i,[2]}$ implies that the second execution chunk $C_{i,[2]}$ is constant for all the jobs. At the end of each execution chunk a time wait function called `ConsumeTime()` is inserted to emulate real execution time advancing the virtual time. An example of task body code is represented in the Fig. 4b. The wait function can either take a constant time or a stochastic variable allowing stochastic task execution time. The timing information (execution time) can be retrieved using any existing technique. The way we model the timing feature of the task allows a decoupling between the real function implementation code and the execution time. This allows the user to either execute the real application code or leave the task body empty, i.e. definition of *abstract tasks*. An abstract task can be defined at early design stages in order to evaluate real-time performances without considering the functional behavior. Although the user can execute the real application code within an execution chunk, its evaluation must be considered as atomic. The execution of the task body does not advance the virtual time of the simulation. Therefore the application code is always executed at zero virtual execution time, while only the `ConsumeTime()` will allow the emulation of the computation time. It follows that the granularity of the execution chunks may depend on the specific application, i.e. the insertion of this function at different granularities enables to verify the system at different level of accuracy. The time wait function is not an atomic function and it can be preempted by the kernel. The granularity of insertion of the `ConsumeTime()` may introduce some approximation in the execution of the code. A preemption by the RTOS kernel can happen almost at any source code instruction (e.g. interrupts). This introduces some complexity on the verification of concurrent systems in presence of shared variables or other synchronization mechanisms. Our preemption model instead only captures the preemption at the level of the `ConsumeTime()` function missing the relation with the functional source code. A possible mitigation is the insertion of the `ConsumeTime()` after every code line, although this may increase the simulation time. It is out of the scope of this paper to do a rigorous timing analysis of the code but it is a topic that we will investigate in a future work.



(a) graphical representation task model.

```
void task1_body(){
    //application code
    consumeTime(C_11)
    BLOCKING_CALL()
    //application code
    consumeTime(C_12)
    BLOCKING_CALL()
    //application code
    .
    .
    consumeTime(C_1n)
    BLOCKING_CALL()
    WAIT_PERIOD()
}
```

(b) task body pseudo-code.

Fig. 4. Computational task model.

3.4 ARINC 653 Model Interface

The ARINC 653 standard specifies an *APplication EXecutive*(APEX) interface which provides a list of services consistent with fulfilling integrated modular avionic platform. The main purpose of the standard is to provide a partitioned environment, both spatially and temporally segregated, where one or more avionics applications can independently execute [2]. In order to implement the APEX services, a specific mapping API (static library) has been developed that implements the ARINC 653 specific interface on top of the generic RTOS API. This library can access methods and attributes of the task model and it wraps the low level generic services exposed by the kernel to satisfy the requirements of the APEX API. In order to perform a SIL simulation for a given task set, the tasks are linked with the mapping library enabling the calling of the APEX services directly within the task body. The API for process management are standard RTOS services for task management (start, resume, suspend, stop) which have mostly a direct mapping with the generic services provided by the RTOS model. The communication API are instead a specialization of the low level communication services provided by the kernel. For example the *queuing ports* defined in the ARINC 653 standard for inter-partition communication are mapped to the concept of port defined in Sect. 3.2 and are specialized in order to model a blocking behavior and a message queuing. Instead the sampling ports are specialized with non-blocking behavior and non-queuing mode. The same approach has been used to map the specialized inter-task communication mechanism (i.e. *buffer* and *blackboard*) with the base communication services provided. An RTOS instance compliant with ARINC 653 comprehend a supervisor and one or more partitions. A partition contains one or more tasks which execute in a temporal and spatial segregated environment. The supervisor, together with the partition scheduler, schedules partitions enforcing temporal segregation while the fixed priority scheduler is used to model the partition-level priority based scheduler defined in the ARINC 653 specification. The structure just described is represented in Fig. 5. This particular instance represent a two-level hierarchi-

cal scheduler as required by the standard ARINC 653. The presented workflow used to integrate the ARINC 653 API is generic. Given a different target RTOS only the mapping between the specific API and the generic services of the RTOS model is needed to enable the SIL simulation.

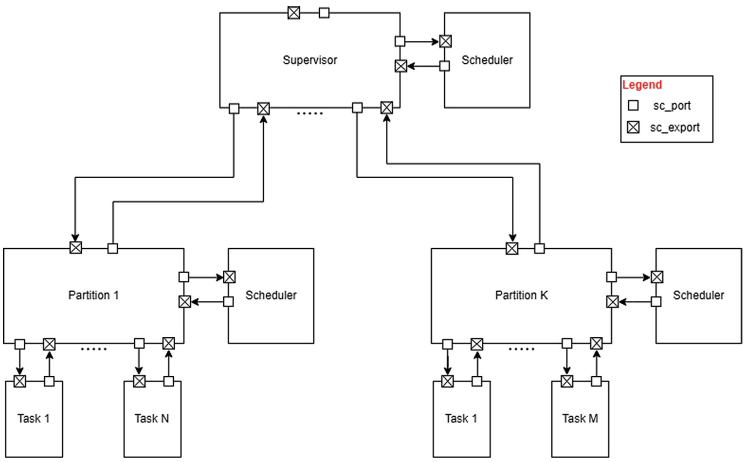


Fig. 5. Specific RTOS model instance compliant with ARINC 653.

4 Uncertainty Analysis

As we introduced before, the modeling of uncertain behavior during the validation and verification of the real-time system is of paramount importance. Modeling of uncertainties is not only useful to capture the real system operation but can also be used to model different design aspects at different validation and verification phases. It can also be used to model unknown platform details at early design stages. For example the execution time of the application (RTOS tasks) can be considered uncertain because the processing unit (e.g. CPU) has not been selected yet at the current design phase. Again the failure probability of physical components is intrinsically non-deterministic and may generates events across the platform that will impact the overall system operation. In the current work we will focus on the probabilistic modeling of the uncertainty affecting real-time operating system, i.e. all the uncertainty behaviors are modeled in terms of probability distributions. In particular we will investigate performances under two main uncertain aspects:

– **Execution time of the task jobs**: this may be the result of the uncertainty at lower architectural levels (e.g. RTOS kernel operations), or application specific uncertainty (e.g. branches in the application code due to unpredictable events).

– **Activation of aperiodic tasks**: the activation of aperiodic tasks can be the result of an internal functional behavior or can be triggered by an external event generated from the environment.

Although our focus is on the above two aspects, our RTOS model is able to handle more specific kernel aspects. For example the model can be easily extended to explicitly model uncertainty affecting the timing of operations such as context switch between jobs or task queuing. The decision on how detailed should be our uncertainty model is a trade-off between modeling effort and probability distribution complexity. The execution time probability distribution is the results of the correlation of multiple uncertainties affecting several operations in both application, kernel and hardware side. Although it is out of the scope of this work an accurate probabilistic modeling of the uncertainty affecting the system, we are currently providing a framework to enable researcher to approach this problem in a systematic way. In the next section we will focus on the two important analysis: *Forward Uncertainty Propagation* [20] and *Statistical Model Checking* [8]. We will see how those two analysis can be used to verify the performances of the system highlighting how the nominal behavior can drastically change when even a small uncertainty is added to the system.

4.1 Forward Uncertainty Quantification

Uncertainty quantification (UQ) [20] is an interdisciplinary area which addresses the problem of quantifying the impact of the uncertainty present in both complex computational and physical models. Two main research areas can be distinguished:

The *Forward Uncertainty Propagation* [20] focuses on assessing some output performances given a probabilistic characterization of the input. A typical application is the evaluation of low-order moments such as mean or variance of some outputs given the probability distribution of the input.

The *Inverse Uncertainty Quantification* [20] instead focuses on the uncertainty characterization of the input given a set of output measurements. This is generally a more challenging problem then forward uncertainty propagation. Although this area is of extreme importance and complementary to the current work, we will not address this topic in the current work.

In the current paper we will focus only on the forward uncertainty propagation analysis. In particular in the next sections we will address the problem of quantifying, in terms of probability distributions, typical performance metrics for a real-time application. There are several efficient UQ techniques available to perform the uncertainty propagation. All those techniques exploit the model properties in order to be more accurate and reduce computation time. Unfortunately given the nature of our model (computational model without any particular structure) we cannot apply any specific technique and we have to rely on a simple Monte Carlo method.

4.2 Statistical Model Checking

Model checking is a method for algorithmic verification of formal systems, i.e. systems based on a formal model representation. The main objective of model checking is to verify whether a given system model possesses certain properties expressed using a specification logic. These properties are usually specified using temporal logic [12], which is an extension of the propositional logic in order to represent realities that change over time. It uses the classic Boolean logic operators to which temporal connectives are added. An improvement to this method is the *Probabilistic Model Checking* that aim to quantify the likelihood for a stochastic system to satisfy some property rather than giving a boolean flag on a property [16]. When dealing with real-time systems it is also important to monitor the timing behavior of the system (when a particular situation arise), hence the possibility to have timing features in the specification of the requirements is also needed [16]. One major problem with MC-based approaches is the state-space explosion problem [8]. *Statistical Model Checking* (SMC) is an approach that has recently been proposed as an alternative to avoid the state explosion problem of probabilistic (numerical) model checking. The approach can be divided in three macro step: (1) the system under test is simulated, (2) the simulation is monitored in order to retrieve relevant parameters, (3) statistical tools, like sequential hypothesis testing or Monte Carlo simulation, are used in order to decide whether the system satisfies the property or not with some degree of confidence [16]. Statistical Model Checking is a trade-off among testing and traditional model checking procedures. The simulation-based approach is less memory and time consumptive than the formal ones, and it is often the only option. To estimate probabilities, SMC uses a number of statistically independent stochastic simulation traces of a discrete event model [13]. In the next sections we will focus on two main methods for the statistical model checking: Monte Carlo and Hypothesis Testing. The *Monte Carlo Method* in the SMC literature [18] is considered a *quantitative method* and it answers to the following question: *What is the probability p for the model under test to satisfy a logic proposition* Ψ*?*; The tool returns a simulation trace for the interesting variables with the number of performed simulations and the number of positive simulations, i.e. the number of simulations in which the property Ψ has satisfied. From these two values the probability of satisfying the requirement is obtained. Instead of manually setting the number of simulations, it is possible to use the Chernoff-Hoeffding bound which provides the minimum number of simulations required to ensure the desired confidence level. The implementation of the Chernoff-Hoeffding bound is based on two parameters: the error margin (ϵ) and the confidence bound (δ). The *Sequential Hypothesis Testing* instead is considered a *qualitative approach* and addresses the following question: *Is the probability p for the model under test to satisfy the property* Ψ *greater or equal than a certain threshold* θ*?* [18]. The hypothesis testing is used to infer if the simulated execution traces provide statistical evidence on the satisfaction or violation of a property. The test is parameterized by three bounds, α and β and the

indifference region IR [18]. In the next section we will see how to take advantage of those methods in order to verify typical RTOS performance metrics.

5 ARINC 653 Application Model: Data Acquisition System

To demonstrate our capability to simulate and verify real-time applications on our RTOS model, we will use a surrogated application targeting an RTOS compliant with the ARINC 653 specification. The sequence diagram of the application is depicted in Fig. 6. It is a simple model of a centralized data acquisition system where all the data are acquired and manipulated on a single computational unit. A task $i \in [0, N]$ in a partition $j \in [A, B, C]$ is represented by τ_i^j. The system is composed by three partitions P_A, P_B and P_C. The partition P_A contains the task τ_1^A that implements all the decision making logic. It is in charge of triggering data acquisition (triggered due to some internal state or due to external events) and based on the manipulated data takes some actions. The task τ_1^A is aperiodic and its execution time is usually variable depending on the sensor measurements and decision making logic. The tasks τ_1^B and τ_2^B in the partition P_B perform a data conditioning and manipulation. Moreover they are in charge of triggering the sensor measurement acquisition from the tasks in the partition P_C. The execution time of the tasks in P_B are slightly variable depending on the amount of data to process at each acquisition event. Tasks in P_C performing the acquisition are usually highly predictable with very short execution time. In our model the generation of the external event is emulated by an aperiodic "artificial" task τ_0^A. The task τ_0^A will trigger the execution of τ_1^A based on a given probability distribution detailed in the next section. The presented base application model can be proportional scaled in order to obtain a more complex system and to analyze the interleaving of the task inside the partitions. In the analysis performed in the next sections we will consider four instances of the model running on the same RTOS. The final system is composed by the task set detailed in the Table 1. The partition has a major time frame of $t_{MTF} = 18\,\text{ms}$ while the time windows have the following configuration $\phi_{A1} = 0\,\text{ms}$, $\phi_{A2} = 12\,\text{ms}$, $t_{A1} = 1\,\text{ms}$, $t_{A2} = 6\,\text{ms}$, $\phi_{B1} = 1\,\text{ms}$, $\phi_{B2} = 7\,\text{ms}$, $t_{B1} = 1\,\text{ms}$, $t_{B2} = 5\,\text{ms}$, $\phi_C = 2\,\text{ms}$, $t_C = 5\,\text{ms}$ where ϕ_{jk} and t_{jk} represent the offset and duration of the k-th time window of the j-th partition. A visual representation of the partition scheduling is presented in Fig. 7. For what is concerns the overall system performances we require that the time elapsed between the data acquisition event (activation of τ_1^A) and the resulting action (end of execution τ_1^A) is not more than 30 ms. This correspond with the response time of tasks in P_A. Of course the tasks in P_A are blocked by all the tasks in the other partitions. The response time of the tasks in P_A will account any delay affecting the tasks in the other partitions.

All the inter-partition communication has been implemented using the *queuing ports*, i.e. each task waits on the queuing port read call until data is available to be manipulated or, if specified, until a timeout expires. Tasks in P_A wait indefinitely on a read call (i.e. until data is available) while task in P_B and P_C have a

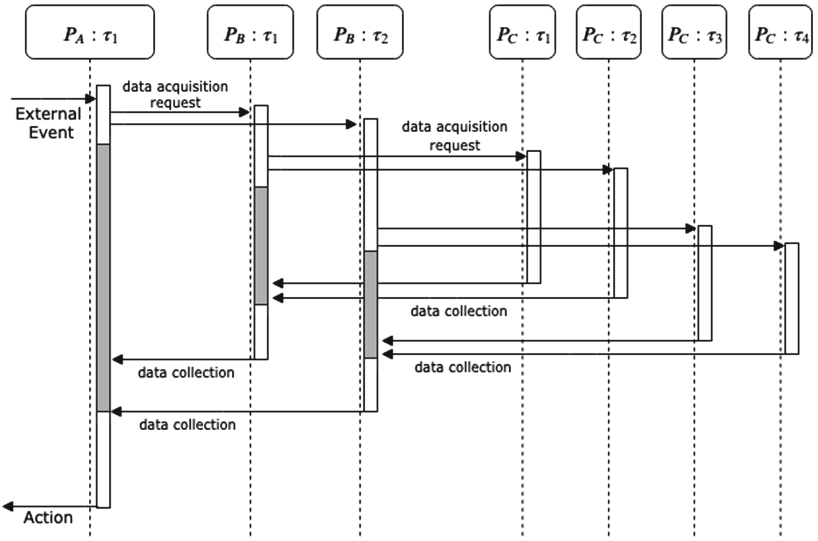


Fig. 6. Application Model - UML-like Sequence Diagram. The grey area represent the task blocking time.

Table 1. Application model task set.

Partition	#Task	Type	Period [ms]	Deadline [ms]	Execution time [ms]
A	4	Aperiodic	-	30	$C^A_{i,[1]} = 0.075$; $C^A_{i,[2]} = 1.075$
B	8	Periodic	18	18	$C^B_{i,[1]} = 0.1$; $C^B_{i,[2]} = 0.55$
C	16	Periodic	18	18	$C^C_i = 0.25$

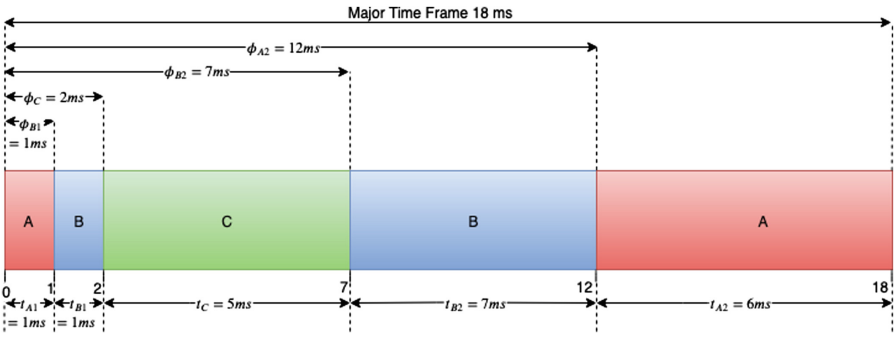


Fig. 7. Partition windows scheduling.

timeout specified within the read call in order not to miss the deadline. Hence, if tasks in P_A don't send the triggering message, the tasks wait only for the specified timeout then waiting for their next release point.

5.1 Application Stochastic Task Model

At the beginning of this section we introduce the task model for all the tasks running on the RTOS model. In the current section we introduce a stochastic variability for some application tasks. In the Fig. 8 is depicted the task model for all the tasks in the application. The tasks in the partition A and B are composed by two execution chunks. The chunks are delimited by the inter-partition communication (possibly blocking calls) involving tasks running on other partitions. The task jobs in the partitions B and C are deterministic and constant during the application execution. The execution time of the generic task instance in B is given by $C^B_{i,j} = C^B_{i,j[1]} + C^B_{i,j[2]} = C^B_{i,[1]} + C^B_{i,[2]}$ where we removed the index j since the execution time is fixed among all the task jobs. The execution of the task jobs running in P_C are constant and given by C^C_i. The execution times for all the tasks are summarized in the Table 1. The tasks running in P_A have a stochastic task model as described below:

- **Uncertain execution time for the task τ^A_i**: The model of the task is represented by two execution chunks. The first one is $C^A_{1,[1]}$ and it is constant for all the job instances. The second chunk $\hat{C}_{1,j[2]}$ is a stochastic variable drawn from a Normal distribution [10], i.e. $\hat{C}_{1,j[2]} \sim \mathcal{N}(\mu = 1, \sigma^2 = 0.25)$. The total execution time for the job instance j is stochastic and it will be the sum of both of the contributions, i.e. $\hat{C}^A_{i,j} = (C^A_{1,[1]} + \hat{C}_{i,j[2]}) \sim \mathcal{N}(\mu + C^A_{1,[1]}, \sigma^2)$.
- **Uncertain activation of the aperiodic task τ^A_i**: As we have seen before, the activation of the task τ^A_i generates a data acquisition event that spreads

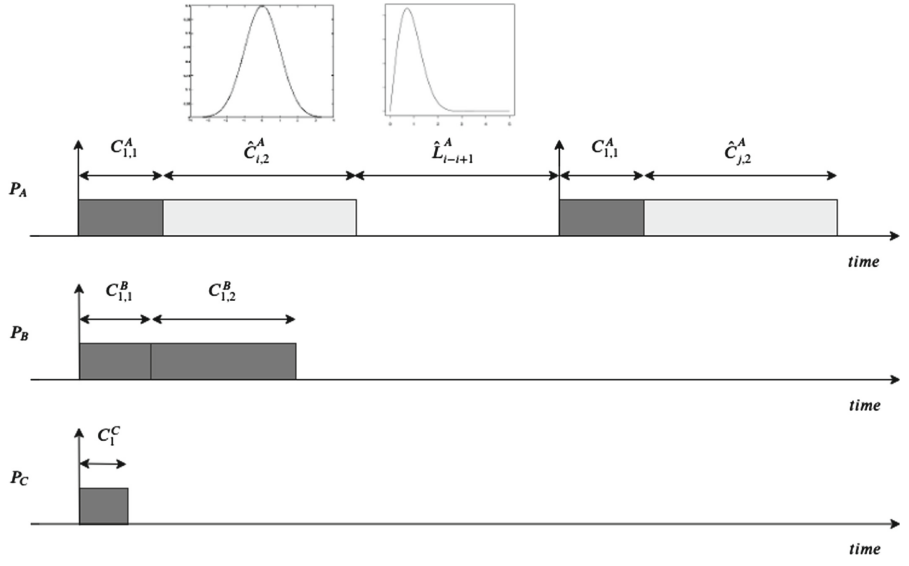


Fig. 8. Application stochastic task model.

across all the system. The probabilistic model is specified in terms of time elapsed between the end of the execution of the task and the next task activation. In the Fig. 8 this value is denoted with L^A_{i-i+1} that is the time elapsed between the end of the execution of the job instance i and the activation of the job instance $i+1$. This variable is a stochastic variable drawn from a Weibull distribution [10], i.e. $L^A_{i-i+1} \sim \mathcal{W}(\lambda = 4, k = 2)$. Choosing this distribution we consider more likely activation triggered around 3 ms after the end of current job execution ($L^A_{i-i+1} \approx 3\,\text{ms}$). The distribution considers also more likely event that are not more than 10 ms distant each other, and it penalizes events that are very close each other, i.e. $L^A_{i-i+1} \approx 0\,\text{ms}$.

6 Results

Given the application described in the Sect. 5, we performed both forward uncertainty propagation and statistical model checking analysis. Although the two analyses are similar, they provide very different insight regarding the model under verification. In particular as we will point out in the next sections the forward uncertainty propagation gives back a global property of the stochastic model. On the other side the SMC predicates on a limited time horizon, therefore nothing can be assessed for the model globally.

6.1 Forward Uncertainty Propagation

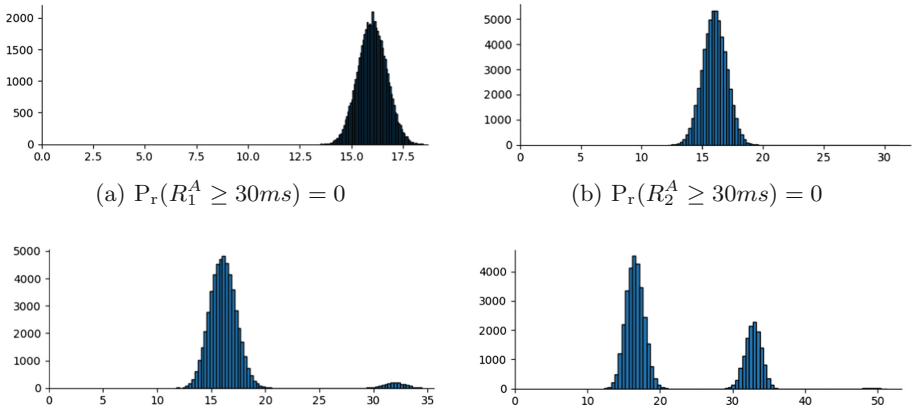
As introduced in the previous sections, the main objective of the forward uncertainty propagation is to quantify the impact of any input uncertainty on the output performances. In this case we considered as input disturbance the probability distributions for both the aperiodic task activation and task execution time for all the four tasks in P_A. As output performance we will focus on the response time of the tasks in P_A denoted as $R^A_{i\in\{1,2,3,4\}}$. The main objective is to verify that their value do not exceed a maximum value fixed to $R_{max} = 30\,\text{ms}$. Although we will focus on the response time our approach is general and can be extended to any relevant RTOS or application performance measure. A qualitative approach has been used to select an appropriate simulation time. It has been tuned in order to exhaustively exercise all the output probability distributions. We selected 1000 s as simulation time frame. A simulation trace for the model has been recorded while the random values for both the Normal and Weibull distribution were generated using the C++ random library [10].

In order to have a reference baseline, the system has been initially simulated in its nominal condition (deterministic simulation) without any stochastic behavior. The nominal case has been designed to satisfy the system requirements. In addition three other different scenarios were simulated considering first the impact of single uncertainties, then the impact of their interaction.

Nominal (Deterministic) Case: In this case the model is purely deterministic. The execution times are fixed equal to the nominal value as detailed in the Table 1. The activation of the aperiodic tasks is triggered at the beginning of

the first time window P_{A1}. The values of the response times for the tasks are $R_1^A = 13.075$, $R_2^A = 14.15$, $R_3^A = 15.225$, $R_4^A = 16.03$.

Stochastic Task Execution: During this use case we will consider only the effect of a stochastic execution time for all the tasks in the partition A. The stochastic characteristic is modeled with the Normal distribution as detailed in Sect. 5.1. In the Fig. 9 are reported the histograms of the response times for all the four tasks $\tau^A_{i\in\{1,2,3,4\}}$ with the probability to miss the response time bound, i.e. $\mathrm{P_r}(R_i^A \geq 30\,\mathrm{ms})$. It can be noted that the response time of task with higher priority (R_1^A) have a normal distribution equal to the nominal execution time $\hat{C}_1^A$ since it doesn't suffer the interference from the other tasks. It never violates the $R_{max} = 30\,\mathrm{ms}$ bound, i.e. $\mathrm{P_r}(R_1^A \geq 30\,\mathrm{ms}) = 0$. The lower priority tasks show an interesting behavior. The response time distribution in Fig. 9c and d present two evident distinct peaks (local maxima) separated by a gap. This distribution resemble a bimodal distribution, i.e. a probability distribution with two different modes. The first peak on the left represents the stochastic variability of the response time around its nominal value. The second peak on the right is the result of task activation not triggered inside the P_A time window but in another partition (P_B or P_C) where they are not allowed to execute. This introduce a delay that is equal to the time needed until the next P_A time windows is available again.

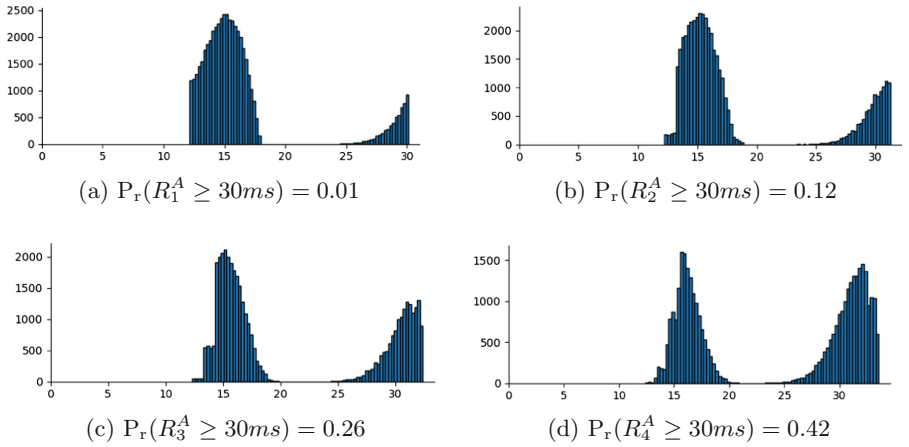


(a) $\mathrm{P_r}(R_1^A \geq 30ms) = 0$ (b) $\mathrm{P_r}(R_2^A \geq 30ms) = 0$

(c) $\mathrm{P_r}(R_3^A \geq 30ms) = 0.03$ (d) $\mathrm{P_r}(R_4^A \geq 30ms) = 0.31$

Fig. 9. Response time histogram for all the tasks in P_A with stochastic execution. In the y-axis we represent the number of occurrences, while in the x-axis the response time in ms.

Stochastic Task Activation: In this case the sequence of activation times for the aperiodic tasks is generated according to a Weibull distribution detailed in Sect. 5.1. The response time histograms of the results are represented in Fig. 10.



(a) $\mathrm{P_r}(R_1^A \geq 30ms) = 0.01$ (b) $\mathrm{P_r}(R_2^A \geq 30ms) = 0.12$

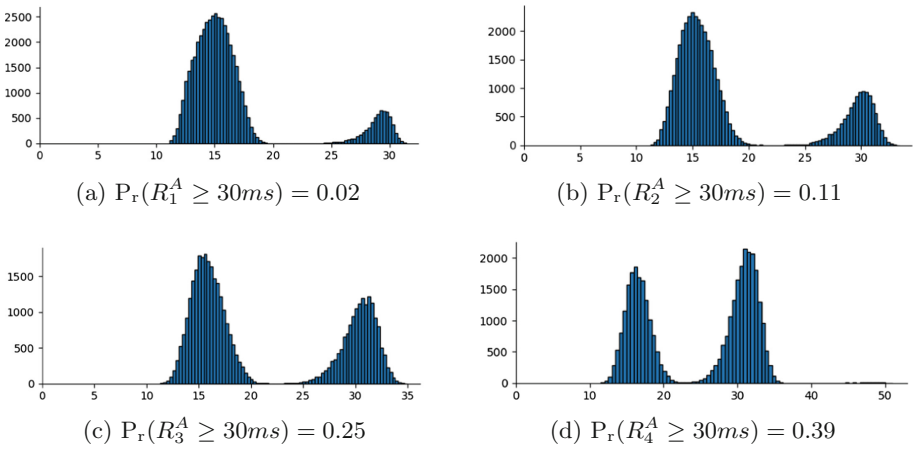
(c) $\mathrm{P_r}(R_3^A \geq 30ms) = 0.26$ (d) $\mathrm{P_r}(R_4^A \geq 30ms) = 0.42$

Fig. 10. Response time histogram for all the tasks in P_A with stochastic activation. In the y-axis we represent the number of occurrences, while in the x-axis the response time in ms.

It can be noted that the stochastic activation time has a significant impact on the response time showing a bimodal distribution for all the tasks. With random activations, all the aperiodic tasks miss some deadlines (i.e. $\mathrm{P_r}(R_i^A \geq 30\,\mathrm{ms}) > 0$) and not only the lower priority tasks. All the task response times are highly impacted by the application partition scheduling represented in Fig. 7. The distribution of the higher priority task τ_1^A depicted in Fig. 10a is limited by a lower bound ($\approx 12\,\mathrm{ms}$) and an upper bound ($\approx 30\,\mathrm{ms}$). The task τ_i^A is divided in two main execution chunk. During the first chunk the event acquisition is spread across all the system. Once all the data are available (τ_i^B and τ_i^C finish their execution) the second execution chunk can start its execution. When the activation of τ_i^A is triggered within the first partition windows P_{A1}, the execution always ends inside the second time window P_{A2}. The resulting response time, in case of activation in P_{A1} is lower-bounded by $C^A_{1,[1]} + t_{B1} + t_C + t_{B2} + C^A_{1,[2]} = 12.15\,\mathrm{ms}$. The worst case activation generating the worst case response time happens when task τ_1^A cannot finish the execution of the first chunk $\tau^A_{1,1}$ within the time window P_{A1}. In this case the data acquisition event is spread in P_{A2} (when the task is again allowed to execute) and a full time frame must be waited in order to get all the data. The upper bound on the execution time is then approximately $t_{MTF} + t_{B1} + t_C + t_{B2} + C^A_{1,[2]} = 30.075\,\mathrm{ms}$. For the other tasks it is possible to do similar considerations even if the behavior is more complicated due to the possible preemption of the higher priority tasks. Similar observations can be made for the lower priority tasks showing similar "steps" in their distributions (especially for τ_2^A depicted in Fig. 10b and for τ_3^A depicted in Fig. 10c). Those steps are the result of the activation of an higher priority task outside its time window while the lower priority task is activated within its time

window, thus managing to complete the execution before the higher priority tasks (since the latter is blocked). This situation is very unlikely so the "steps" in the distributions are small. For τ_2^A the step is only one because it has only one higher priority task τ_1^A, while for τ_3^A the steps are two since it has two higher priority tasks (τ_1^A and τ_2^A) with the second step much lower because in this case both τ_1^A and τ_2^A must have an unfavorable activation value (the product of two probabilities is less than the single probabilities).

Stochastic Task Execution and Activation: In this use case we consider both the execution and activation uncertainties. The distributions are the same considered in the previous use cases. When injecting both uncertainties in the system, the output distributions are similar to the case with only stochastic activation but with the response time covering a wider range of values due to the introduction of variable execution time. By introducing both uncertainties, one would expect a degradation of the system performance by an increasing of deadline miss and an equivalent reduction in the probability to meet the imposed requirement. However the mixing of both uncertainties can lead to a small improvement in the performances as shown in Fig. 11 by the probabilities $\mathrm{P_r}(R_i^A \geq 30\,\mathrm{ms})$; compared to the previous case, the tasks τ_3 and τ_4 present a smaller number of deadlines miss, τ_2 performance are almost unchanged and τ_1 is the most impacted. It is interesting to note that a nominal deterministic simulation may not empathize this kind of behaviors vanishing the overall performance assessment.



(a) $\mathrm{P_r}(R_1^A \geq 30ms) = 0.02$ (b) $\mathrm{P_r}(R_2^A \geq 30ms) = 0.11$

(c) $\mathrm{P_r}(R_3^A \geq 30ms) = 0.25$ (d) $\mathrm{P_r}(R_4^A \geq 30ms) = 0.39$

Fig. 11. Response time histogram for all the tasks in P_A with stochastic execution and activation. In the y-axis we represent the number of occurrences, while in the x-axis the response time in ms.

6.2 Statistical Model Checking

The statistical model checking aims to verify a linear temporal logic targeting a bounded time window, i.e. Bounded Linear Temporal Logic (BLTL). Differently from the forward uncertainty propagation analysis that provided global results, the statistical model checking targets a limited time windows. In our use case we aim to verify the four properties expressed by the BLTL in Eq. 1. The equation expresses the system requirement that: whenever a task τ_i finishes its execution (finish task event $\mathcal{E}_i^f$ is triggered), the resulting response time R_i is always less then a maximum response time R_{max} in the given time window (operator $G_{\leq N}$).

$$\Psi := G_{\leq N}\Big((\mathcal{E}_i^f = 1) => (R_i <= R_{max})\Big) \tag{1}$$

The time interval was chosen equal to four major time frames, i.e. $N = 72$ ms; The maximum response time value was set to $R_{max} = 30$ ms.

To perform the statistical model checking analyses we used the Plasma Lab tool. Plasma Lab is a compact, efficient and flexible platform for statistical model checking of stochastic models [13]. All the simulations have been carried out on a workstation with an i7-8850H CPU @ 2.6 GHz and 32 GB of RAM. The Plasma tool makes available different verification algorithms. We focused on two main algorithms: Monte Carlo Method and Hypotheses Testing. The *Monte Carlo method with Chernoff-Hoeffding* was introduced in Sect. 4.2. Plasma implements this method enabling the setting of the error margin ϵ and the confidence bound δ. We selected $\delta = 95\%$ for the confidence and the precision has ranged on 0.1, 0.05, 0.005 requiring respectively 38, 149, 14889 simulations. The SMC analyses were performed on the model with both task execution time and task activation uncertainty. The results are reported in the Table 2 where p represent the

Table 2. PLASMA analysis with Chernoff bound with 95% confidence bound and variable error margin.

Task	Error margin (ϵ)	# Simulations	p	Simulation time [s]
τ_1^A	0.1	38	1.0	20
	0.05	149	0.966	75
	0.005	14889	0.983	7627
τ_2^A	0.1	38	0.921	20
	0.05	149	0.872	75
	0.005	14889	0.881	7627
τ_3^A	0.1	38	0.737	20
	0.05	149	0.772	75
	0.005	14889	0.733	7627
τ_4^A	0.1	38	0.579	20
	0.05	149	0.51	75
	0.005	14889	0.571	7627

Table 3. PLASMA analysis with hypothesis testing bound.

Task	Goal Probability (p)	# Simulations	Estimate Probability (p')	Simulation Time [s]
τ_1^A	0.9	267	0.977	808
	0.7	171	0.982	722
	0.5	127	0.953	331
τ_2^A	0.9	1527	0.887	808
	0.7	258	0.887	722
	0.5	155	0.871	331
τ_3^A	0.9	114	0.719	808
	0.7	1325	0.737	722
	0.5	201	0.786	331
τ_4^A	0.9	76	0.618	808
	0.7	222	0.482	722
	0.5	809	0.571	331

probability to satisfy the BLTL in Eq. 1. It is interesting to note that despite the introduction of a small uncertainty, the performance of the system drops in a significant way compared to the nominal case.

The *Hypothesis Testing* method was introduced in Sect. 4.2. Plasma implements this method enabling the user to set the probabilistic bounds α and β. Also in this case we focused on the response time of the tasks in P_A. This method aims to verify whether the probability to satisfy the LBTL Ψ in Eq. 1 is less than a probability p. We considered three values for the goal probability $p = \{0.9, 0.7, 0.5\}$, while we selected $\alpha = \beta = IR = 0.01$. The results of the analysis are reported in Table 3. The tool returns the probability of satisfying the requirement. The True/False (Green/Red) indicator informs the user if the set probability goal has been respected or not.

7 Conclusions and Future Work

In the current paper we presented a generic modular RTOS SystemC model. The RTOS model has been designed in order to be customized implementing RTOS specific services. This allows the user to run the real application code on top of the model without modifying the original application code targeting a specific RTOS implementation. The proposed model captures the timing info of the application (execution time) by defining a specific API to be added in the application code. Furthermore the model also allows the modeling of uncertain behavior. We presented a use case application where the execution time of the tasks and their aperiodic activation is modeled by mean of probability distributions. The performance analysis of the stochastic application was performed with two different methods: forward uncertainty propagation and statistic model checking. The proposed analyses highlighted how a small uncertainty may introduce a consistent performance degradation. This shows that analyzing the system under uncertain conditions is of paramount importance to a proper verification assessment. Different aspects were not investigated that are of great importance. Claiming generality of the RTOS model is quite difficult since any COT RTOS may implement specific services. Future activity will validate the

model against common RTOSes (FreeRTOS, VxWorks, DEOS) and standardized RTOS interfaces as OSEK OS and POSIX. The RTOS model could be enhanced in order to enable the modeling of multicore scheduling. The model can also be improved in order to integrate hardware models (FPGA, FLASH memory, EEPROM, etc.) using transaction level modeling (TLM). This will enable an HW/SW co-simulation in a unified simulation framework. Moreover in order to ease the simulation of application code, techniques for automatic back annotation will be investigated. In addition the performance of real-time systems are affected by uncertainty at different levels such as hardware, RTOS kernel or application. On one side, modeling the uncertainty at all the levels may be time consuming with the estimation of multiple probability distribution. On the other side considering the cumulative distribution of many uncertainties at different levels may move the complexity to fewer distributions with very high modeling complexity. An active research area called Inverse Uncertainty Quantification addresses this issue although, to the best of the authors knowledge, there are not existing works targeting complex embedded systems.

Acknowledgments. This work has received funding from the Clean Sky 2 Joint Undertaking under the European Union's Horizon 2020 research and innovation programme under grant agreement N° 807081.

References

1. Accelera: Core SystemC Language. https://www.accellera.org/downloads/standards/systemc. Accessed Aug 2019
2. Airlines Electronic Engineering Committee (AEEC): ARINC Specification 653 P1. Avionics application software standard interface (2010). rev. 3
3. Barry, R.: FreeRTOS. http://freerots.org. Accessed Aug 2019
4. Buttazzo, G.: Research trends in real-time computing for embedded systems. SIGBED Rev. **3**(3), 1–10 (2006). https://doi.org/10.1145/1164050.1164052
5. D'Angelo, M., Ferrari, A., Ogaard, O., Pinello, C., Ulisse, A.: A Simulator based on QEMU and SystemC for robustness testing of a networked linux-based fire detection and alarm system. In: Proceedings of the Conference on Embedded Real Time Systems and Software, pp. 1–9 (2012)
6. DDC-I: DEOS. https://www.ddci.com/category/deos/. Accessed Aug 2019
7. Grimm, C., Rathmair, M.: Dealing with uncertainties in analog/mixed-signal systems: invited. In: Proceedings of the 54th Annual Design Automation Conference 2017, New York, NY, USA, pp. 35:1–35:6 (2017)
8. Hansen, J.P., Wrage, L.: Verification of real-time systems using statistical model checking. In: AIAA Infotech@ Aerospace, p. 1866 (2015)
9. Huck, E., Miramond, B., Verdier, F.: A modular SystemC RTOS model for embedded services exploration. In: Proceedings of First European Workshop on Design and Architectures for Signal and Image Processing (2007)
10. ISO International Standard ISO/IEC 14882:2017(E): Programming Language C++: Random Library. http://www.cplusplus.com/reference/random/. Accessed Aug 2019

11. Keutzer, K., Newton, A.R., Rabaey, J.M., Sangiovanni-Vincentelli, A.: System-level design: orthogonalization of concerns and platform-based design. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **19**(12), 1523–1543 (2000)
12. Lamport, L.: The temporal logic of actions. ACM Trans. Program. Lang. Syst. (TOPLAS) **16**(3), 872–923 (1994)
13. Legay, A., Sedwards, S., Traonouez, L.M.: Plasma lab: a modular statistical model checking platform. In: Margaria, T., Steffen, B. (eds.) ISoLA 2016. LNCS, vol. 9952, pp. 77–93. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-47166-2_6
14. Meyerowitz, T., Sangiovanni-Vincentelli, A., Sauermann, M., Langen, D.: Source-level timing annotation and simulation for a heterogeneous multiprocessor. In: 2008 Design, Automation and Test in Europe, pp. 276–279 (2008). https://doi.org/10.1109/DATE.2008.4484897
15. Mignogna, A., Ferrante, O., Carloni, M., Ferrari, A.: A fully configurable RTOS model for large scale distributed embedded systems simulations based on SystemC. In: Proceedings of Conference on Applied Simulation and Modelling. ACTA Press (2011)
16. Plasma-Lab: Statistical Model Checking. https://project.inria.fr/plasma-lab/statistical-model-checking/. Accessed Aug 2019
17. Posadas, H., Ádamez, J., Villar, E., Blasco, F., Escuder, F.: RTOS modeling in SystemC for real-time embedded SW simulation: a POSIX model. Des. Autom. Emb. Syst. **10**, 209–227 (2005)
18. Quilbeuf, J., Cavalcante, E., Traonouez, L.-M., Oquendo, F., Batista, T., Legay, A.: A logic for the statistical model checking of dynamic software architectures. In: Margaria, T., Steffen, B. (eds.) ISoLA 2016. LNCS, vol. 9952, pp. 806–820. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-47166-2_56
19. Segala, R.: Modeling and verification of randomized distributed real-time systems. Ph.D. thesis, Massachusetts Institute of Technology (1995)
20. Smith, R.C.: Uncertainty Quantification: Theory, Implementation, and Applications. SIAM, Philadelphia (2013)
21. Swan, S.: An introduction to system level modeling in SystemC 2.0. Cadence Design Systems Inc., draft report (2001)
22. Wilhelm, R., et al.: The worst-case execution-time problem - overview of methods and survey of tools. ACM Trans. Embed. Comput. Syst. (TECS) **7**(3), 36 (2008)
23. Wind Rivers Systems: VxWorks. https://www.windriver.com/products/vxworks/. Accessed Aug 2019
24. Zabel, H., Müller, W., Gerstlauer, A.: Accurate RTOS modeling and analysis with SystemC. In: Ecker, W., Müller, W. (eds.) Hardware-Dependent Software: Principles and Practice, pp. 233–260. Springer, Heidelberg (2009). https://doi.org/10.1007/978-1-4020-9436-1_9
25. Zhang, M., Ali, S., Yue, T., Nguyen, P.: Uncertainty modeling framework for the integration level v. 1. Simula Research Laboratory (2016)
26. Zhang, M., Ali, S., Yue, T., Norgren, R., Okariz, O.: Uncertainty-wise cyber-physical system test modeling. Softw. Syst. Model. **18**(2), 1379–1418 (2019)